Multiobjective Optimization with Pretrained AIAntenna Models

This demo optimizes a 5.9 GHz patch microstrip antenna on a Teflon substrate for competing objectives — bandwidth and gain — using a pretrained artificial intelligence (AI) surrogate. The scenario is a vehicle-to-infrastructure (V2I) roadside unit for the 5.9 GHz vehicle-to-everything (V2X) band. The link budget demands maximum gain for highway-distance coverage, while wider bandwidth provides margin for manufacturing tolerances and multi-channel operation. The Teflon substrate provides thermal stability in an outdoor enclosure.

Bandwidth and gain compete in a patch antenna: the geometric parameters that maximize one do not simultaneously maximize the other, particularly under a resonance constraint. Mapping this trade-off with full-wave electromagnetic (EM) simulation could take days — each evaluation requires several minutes, and the optimizer needs hundreds. An AI surrogate evaluates in a fraction of a second, making the full multiobjective optimization (Pareto front) exploration practical.

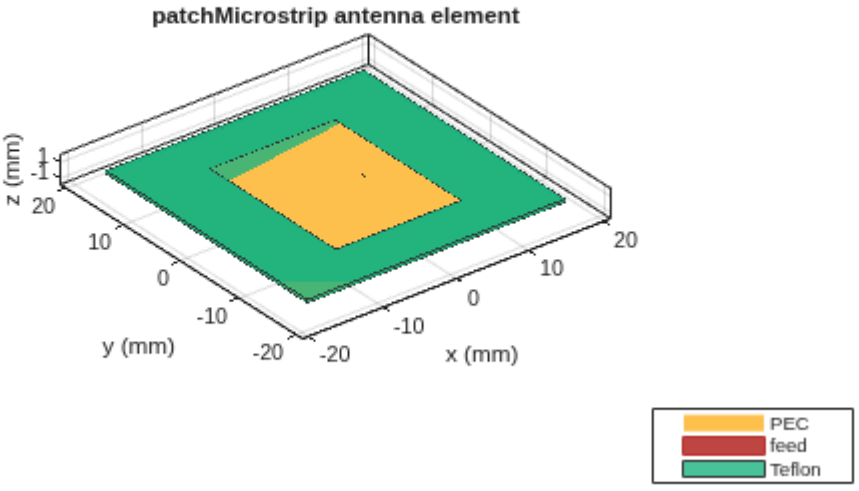
Design the AI Antenna

Create the baseline design at 5.9 GHz. The `ForAI=true` flag returns an `AIAntenna` object backed by pretrained AI models instead of running full-wave simulation.

```
aiant =
design(patchMicrostrip(Substrate=dielectric("Teflon")),5.9e9,ForAI=true);
```

Visualize the geometry.

```
show(aiant)
```



Evaluate the initial design point — this is the baseline before optimization.

```
bw_AI0 = bandwidth(aiant)/1e6;
rad_AI0 = peakRadiation(aiant,5.9e9);
disp("Initial design: BW = " + round(bw_AI0,1) + " MHz, Gain = " +
round(rad_AI0,2) + " dBi")
```

```
Initial design: BW = 51 MHz, Gain = 7.82 dBi
```

Inspect the tunable parameter ranges. The AI model is trained and accurate within these bounds.

```
tr = tunableRanges(aiant)
```

```
tr = 4×4 table
```

...

	Length		Width		Height		SubstrateEpsilonR
	min	max	min	max	min	max	min
1 resonantFrequency	0.0145	0.0196	0.0186	0.0252	2.9804e-04	4.0323e-04	1.7850
2 bandwidth	0.0145	0.0196	0.0186	0.0252	2.9804e-04	4.0323e-04	1.7850
3 beamwidth	0.0145	0.0196	0.0186	0.0252	2.9804e-04	4.0323e-04	1.7850
4 peakRadiation	0.0145	0.0196	0.0186	0.0252	2.9804e-04	4.0323e-04	1.7850

Set Up Multiobjective Optimization

Define the optimization problem: maximize both bandwidth and peak radiation by varying Length, Width, and Height. SubstrateEpsilonR is fixed at the Teflon default — the material is a design choice, not a free variable. The bounds come from `tunableRanges`, inset slightly to avoid boundary effects. A nonlinear constraint keeps the resonant frequency within ±2% of the operating frequency, ensuring all solutions on the Pareto front are actually usable at 5.9 GHz.

```
f0 = 5.9e9;
lb = [0.0146, 0.0188, 0.000300];
ub = [0.0194, 0.0250, 0.000402];
objFcn = @(optimInput) multiObjFcnAI(aiant,f0,optimInput);
constrFcn = @(optimInput) constraintFcnAI(aiant,f0,optimInput);
```

Configure `paretosearch` — a direct-search multiobjective optimizer that requires no gradients and is well suited for bounded problems with inexpensive evaluations.

```
opts = optimoptions("paretosearch",ParetoSetSize=20,Display="off");
```

Run the Optimization

Each AI evaluation completes in a fraction of a second, enabling the optimizer to map the full Pareto front in under a minute.

```
rng default
tic
[X,F] = paretosearch(objFcn,3,[],[],[],[],lb,ub,constrFcn,opts);
toc
```

```
Elapsed time is 42.089975 seconds.
```

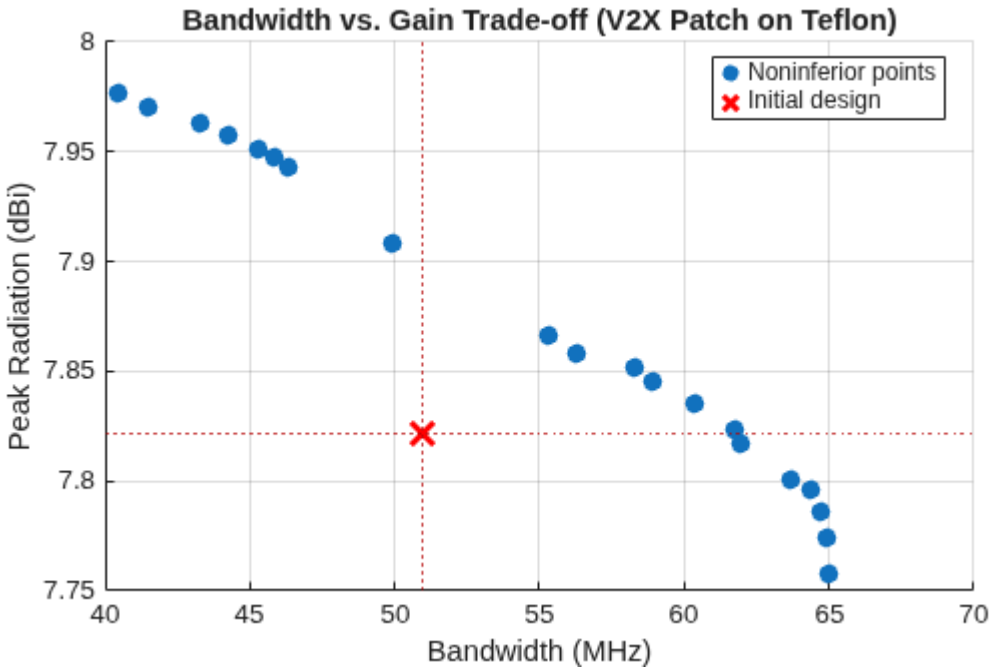
```
BW = -F(:,1)/1e6;
Gain = -F(:,2);
```

The optimizer ran ~500 function evaluations. Each one calls `bandwidth` and `peakRadiation` for the objectives, plus `resonantFrequency` for the constraint. With full-wave EM simulation, each evaluation could require a frequency sweep (~3 minutes) plus a full-sphere far-field computation (~3 minutes), driving the total computational cost to the order of days.

Visualize the Pareto Front

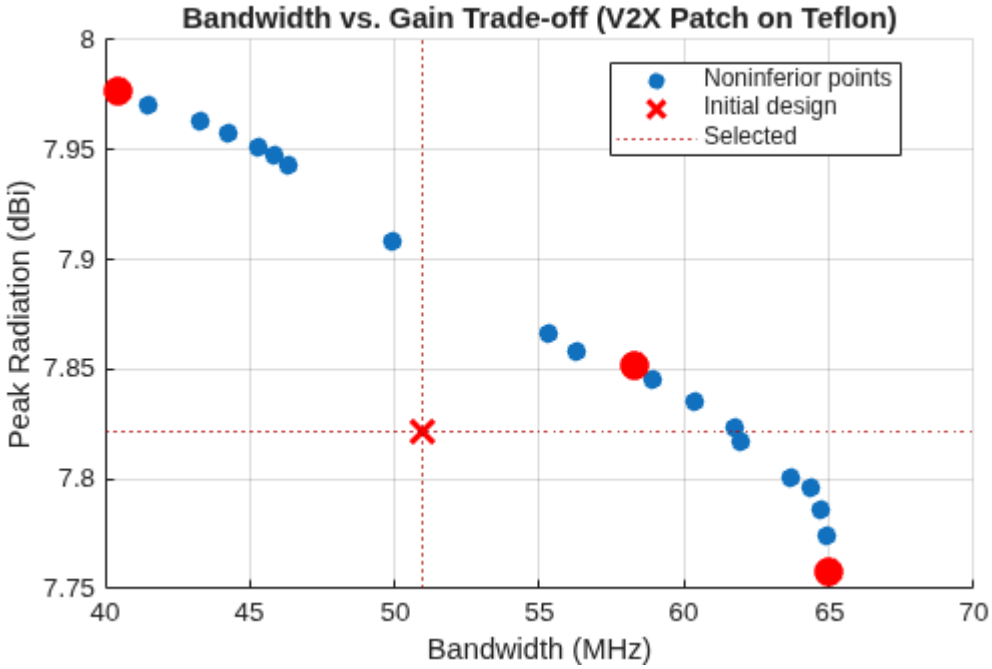
The trade-off curve shows the noninferior points — solutions for which neither objective can be improved without worsening the other. Overlaying the initial `design` point shows how the optimizer improved on the baseline.

```
figure
scatter(BW,Gain,50,"filled")
hold on
plot(bw_AI0,rad_AI0,"rx",MarkerSize=12,LineWidth=2)
xline(bw_AI0,":","Color",[0.7 0 0])
yline(rad_AI0,":","Color",[0.7 0 0])
hold off
xlabel("Bandwidth (MHz)")
ylabel("Peak Radiation (dBi)")
title("Bandwidth vs. Gain Trade-off (V2X Patch on Teflon)")
legend(["Noninferior points","Initial design"],Location="best")
grid on
```



Select three representative solutions: maximum bandwidth, maximum gain, and a balanced compromise. The balanced point is chosen as the one nearest the utopia point (simultaneously maximum bandwidth and maximum gain) in normalized objective space.

```
[~,idxMaxBW] = max(BW);
[~,idxMaxGain] = max(Gain);
dist = vecnorm([BW - max(BW), Gain - max(Gain)]./[range(BW), range(Gain)],
2, 2);
[~,idxBal] = min(dist);
hold on
scatter(BW([idxMaxBW,idxMaxGain,idxBal]),Gain([idxMaxBW,idxMaxGain,idxBal]),1
20,"r","filled")
legend(["Noninferior points","Initial design","Selected"],Location="best")
hold off
```



Inspect the selected designs.

```
selectedDesigns = array2table([ ...
    X(idxMaxBW,:)*1e3, BW(idxMaxBW), Gain(idxMaxBW); ...
    X(idxMaxGain,:)*1e3, BW(idxMaxGain), Gain(idxMaxGain); ...
    X(idxBal,:)*1e3, BW(idxBal), Gain(idxBal)], ...
    VariableNames=["Length (mm)","Width (mm)","Height (mm)","BW (MHz)","Gain
(dBi)"], ...
    RowNames=["Max BW","Max Gain","Balanced"])
```

`selectedDesigns = 3×5 table`

	Length (mm)	Width (mm)	Height (mm)	BW (MHz)	Gain (dBi)
1 Max BW	16.8500	24.5365	0.4020	65.0155	7.7578
2 Max Gain	17.6136	21.9546	0.3000	40.4513	7.9767
3 Balanced	17.1813	22.3688	0.4020	58.2645	7.8515

The frequency constraint keeps all solutions on-band, which limits the extremes the optimizer can reach. The max-bandwidth solution achieves ~65 MHz — a significant improvement over the 51 MHz baseline. The max-gain solution approaches 8 dBi with narrower bandwidth. The balanced solution improves both objectives over the initial design while staying well within the V2X band.

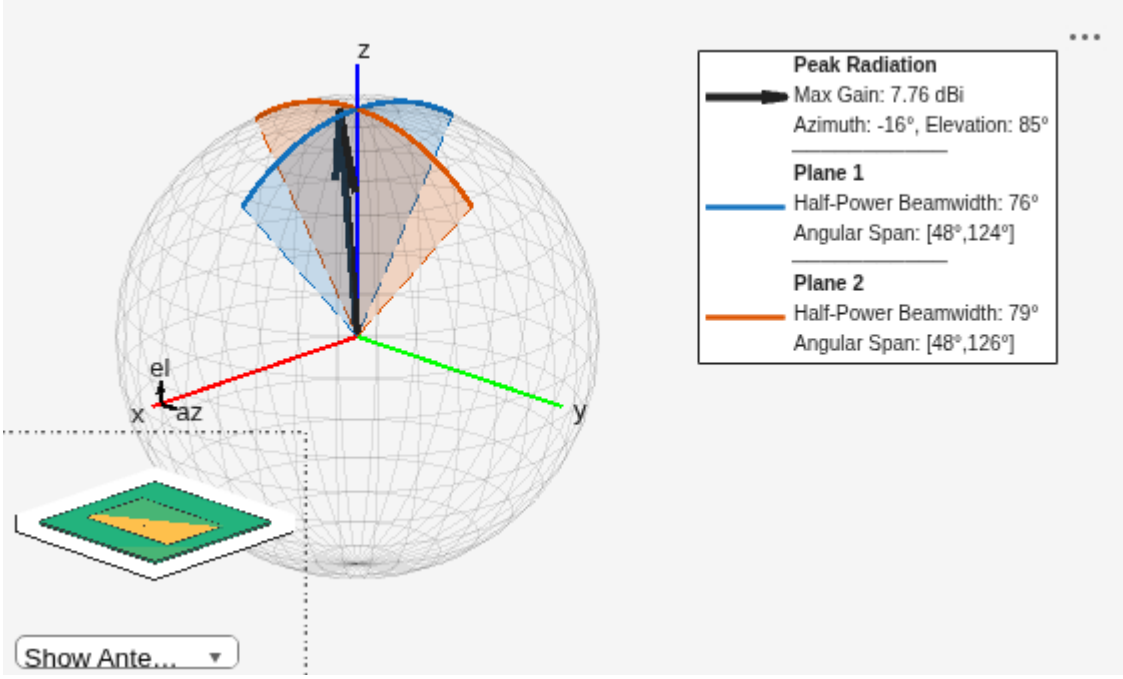
Far-Field Visualization (R2026b)

As of R2026b, calling `beamwidth` or `peakRadiation` without output arguments produces a far-field visualization plot, enabling direct visual comparison of the radiation shape across candidate designs without additional plotting code.

Max-Bandwidth Solution

This solution achieves the broadest passband at the cost of reduced directivity.

```
aiant.Length = X(idxMaxBW,1);
aiant.Width = X(idxMaxBW,2);
aiant.Height = X(idxMaxBW,3);
peakRadiation(aiant,f0)
```

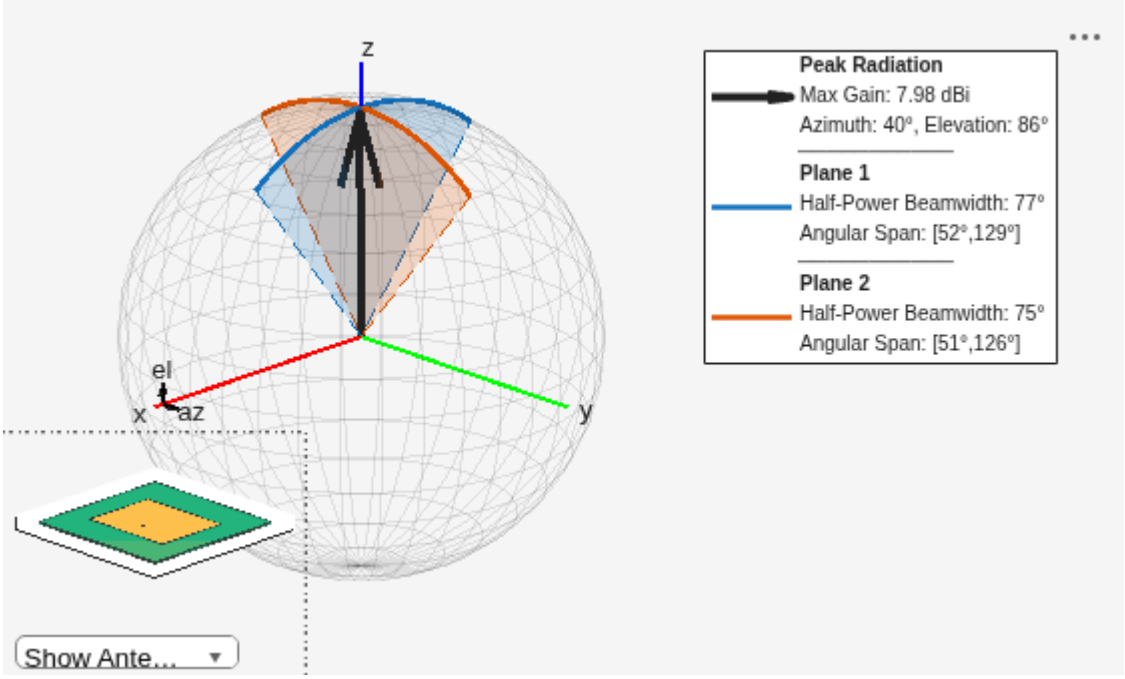



```
ans =
7.7578
```

Max-Gain Solution

This solution achieves the highest directivity at the cost of reduced bandwidth.

```
aiant.Length = X(idxMaxGain,1);
aiant.Width = X(idxMaxGain,2);
aiant.Height = X(idxMaxGain,3);
peakRadiation(aiant,f0)
```

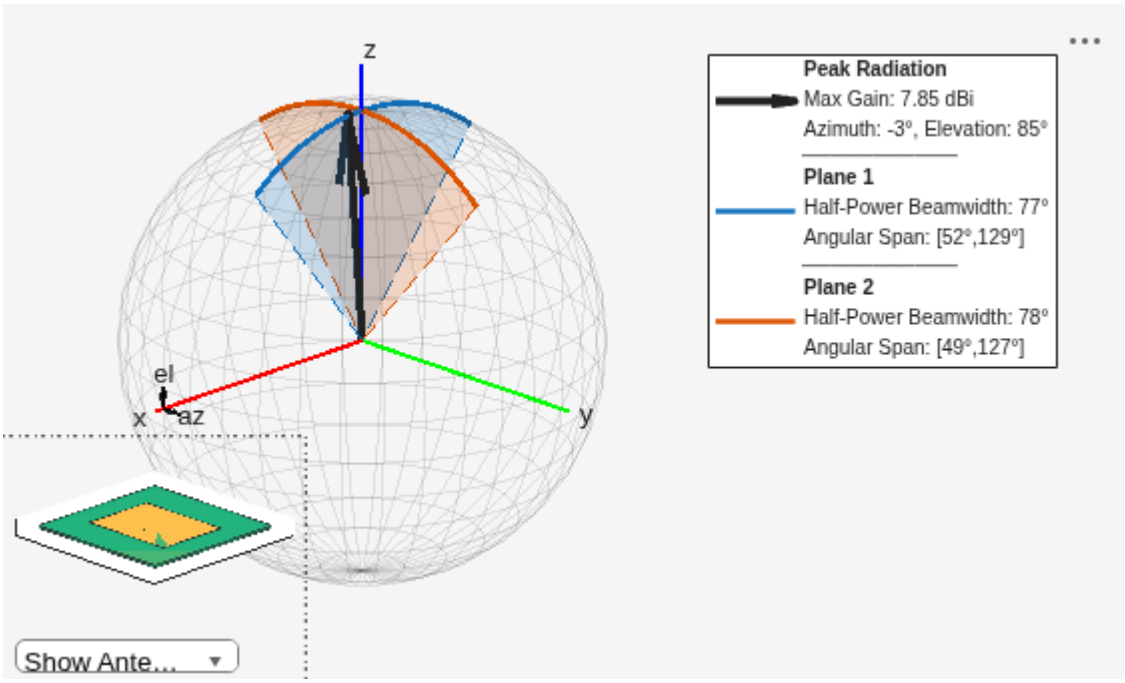



```
ans =
7.9767
```

Balanced Solution

This solution offers a practical compromise, improving both bandwidth and gain over the initial design while staying on-frequency.

```
aiant.Length = X(idxBal,1);
aiant.Width = X(idxBal,2);
aiant.Height = X(idxBal,3);
peakRadiation(aiant,f0)
```



```
ans =
7.8515
```

Full-Wave Validation

Before committing to fabrication, validate the chosen design with full-wave EM simulation. This is the recommended workflow: use AI for rapid design-space exploration, then confirm the finalist with rigorous simulation.

```
switch "Balanced"
    case "Max BW"
        idx = idxMaxBW;
    case "Max Gain"
        idx = idxMaxGain;
    case "Balanced"
        idx = idxBal;
end
aiant.Length = X(idx,1);
aiant.Width = X(idx,2);
aiant.Height = X(idx,3);
pm = exportAntenna(aiant);
freq = linspace(0.7,1.3,1001)*f0;
```

Resonant Frequency

`SweepOption="interp"` fits a rational function to the S-parameters from a sparse set of EM solves, then evaluates at all 1001 frequency points — fine resolution at minimal additional cost.

```
tic
fR_EM = resonantFrequency(pm,freq,SweepOption="interp");
toc
```

```
Elapsed time is 232.214363 seconds.
```

Bandwidth

The bandwidth computation reuses the cached EM solutions from the resonant frequency sweep, so it returns almost instantly.

```
tic
bw_EM = bandwidth(pm,freq,SweepOption="interp");
toc
```

```
Elapsed time is 1.567704 seconds.
```

Beamwidth

Far-field quantities require evaluating the radiation integral at each observation angle — a separate computation from the port analysis. Compute beamwidth at 1-degree resolution in two elevation planes for the patch (az=0 and az=90).

```
tic
hpbw_el0_EM = beamwidth(pm,f0,0,0:360);
toc
```

```
Elapsed time is 7.702464 seconds.
```

```
tic
hpbw_el90_EM = beamwidth(pm,f0,90,0:360);
toc
```

```
Elapsed time is 0.581924 seconds.
```

Peak Radiation

Peak radiation requires a full-sphere scan at 1-degree resolution (361 azimuth x 181 elevation directions).

```
tic
rad_EM = peakRadiation(pm,f0,0:360,-90:90);
toc
```

```
Elapsed time is 92.373137 seconds.
```

Compare AI vs. Full-Wave

Tabulate the AI surrogate predictions against full-wave EM results. The AI models are accurate enough to enable the optimizer to reliably explore the design space, with full-wave simulation providing the final confirmation before fabrication.

```
fR_AI = resonantFrequency(aiant);
bw_AI = bandwidth(aiant);
hpbw_AI = beamwidth(aiant,f0);
rad_AI = peakRadiation(aiant,f0);
```

```
table([fR_AI;bw_AI;hpbw_AI(1);hpbw_AI(2);rad_AI], ...
    [fR_EM(1);bw_EM;hpbw_el0_EM;hpbw_el90_EM;rad_EM], ...
    VariableNames=["AI","EM"], ...
    RowNames=["Resonant Frequency","10-dB Bandwidth", ...
    "3-dB Beamwidth (Plane 1)","3-dB Beamwidth (Plane 2)","Peak Radiation"])
```

ans = 5×2 table

	AI	EM
1 Resonant Frequency	5.8460e+09	5.8576e+09
2 10-dB Bandwidth	5.8265e+07	5.7635e+07

	AI	EM
3 3-dB Beamwidth (Plane 1)	76.9141	78
4 3-dB Beamwidth (Plane 2)	77.9158	74.0000
5 Peak Radiation	7.8515	7.9291

Helper Function

The `multiObjFcnAI` local function evaluates the two competing objectives for a given set of geometric parameters. It assigns the optimization variables to the AIAntenna object and returns the negated bandwidth and peak radiation (negated because `paretosearch` minimizes).

```
function y = multiObjFcnAI(aiant,f0,optimInput)
aiant.Length = optimInput(1);
aiant.Width = optimInput(2);
aiant.Height = optimInput(3);
bw = bandwidth(aiant);
rad = peakRadiation(aiant,f0);
y = [-bw, -rad];
end
```

Constraint Function

The `constraintFcnAI` local function enforces that the resonant frequency stays within ±2% of the operating frequency. This prevents the optimizer from exploiting geometries that achieve high bandwidth or gain by shifting the resonance away from the target band.

```
function [c,ceq] = constraintFcnAI(aiant,f0,optimInput)
aiant.Length = optimInput(1);
aiant.Width = optimInput(2);
aiant.Height = optimInput(3);
fR = resonantFrequency(aiant);
c(1) = -fR + f0*0.98;
c(2) = fR - f0*1.02;
ceq = [];
end
```